

Cluster-Aware Attention-Based Deep Reinforcement Learning for Pickup and Delivery Problems

Wentao Wang^{1†}, Lifeng Han^{1†}, Guangyu Zou^{2*}

¹Leicester International Institute, Dalian University of Technology, Panjin, 124221, Liaoning, China.

²Department of Electronic and Information Technology, Dalian University of Technology, Panjin, 124221, Liaoning, China.

*Corresponding author(s). E-mail(s): zgy_neu@hotmail.com;
Contributing authors: shiy anxil@mail.dlut.edu.cn;
2369787465@mail.dlut.edu.cn;

[†]These authors contributed equally to this work.

Abstract

The Pickup and Delivery Problem (PDP) is a challenging routing problem characterized by coupled pickup–delivery pairs, precedence constraints, and spatial structures that may exhibit clustered patterns. Existing DRL-based PDP solvers usually represent instances as flat node graphs, while search-augmented neural methods often improve solution quality through iterative inference-time refinement, leading to increased latency. We propose Cluster-Aware Attention-based Deep Reinforcement Learning (CAADRL), a neural construction policy that explicitly models multi-scale pickup–delivery structure through cluster-aware representation learning and hierarchical decoding. The proposed architecture separates local intra-cluster routing preferences from global inter-cluster transition preferences, while maintaining route feasibility through precedence-aware dynamic masking. Experiments on 10,000-instance clustered and uniform PDP benchmarks show that CAADRL provides an effective and efficient inductive bias for constrained neural routing. Under the Sampling-1280 decoding setting, CAADRL reduces the paired objective over the Heterogeneous Attention baseline by 27.37% on PDP40-clustered and 47.42% on PDP80-clustered. On PDP80-uniform, CAADRL with Sampling-12800 achieves a 7.49% lower mean objective than the neural collaborative-search baseline NCS with 3000 improvement iterations, while avoiding iterative ruin-and-repair refinement. Runtime

evaluation further shows sub-0.20-second per-instance inference on PDP80 in the main benchmark setting, highlighting the potential of CAADRL for scalable and low-latency PDP inference.

Keywords: Pickup and Delivery Problem, Deep reinforcement learning, Cluster-aware attention, Hierarchical decoding, Neural combinatorial optimization

1 Introduction

The Pickup and Delivery Problem (PDP) is a central routing problem in transportation and logistics, with applications in shared mobility, city-scale delivery, crowdsourced logistics, online dispatching, and electrified fleets [2, 10, 19, 23]. Unlike the classical Traveling Salesman Problem (TSP) or many standard Vehicle Routing Problem (VRP) variants, each PDP request consists of a pickup node and a corresponding delivery node that must be served by the same vehicle under a strict precedence constraint [27]. This pickup–delivery coupling makes route construction substantially more difficult: a feasible policy must not only choose short edges, but also preserve pairwise precedence, avoid invalid delivery actions, and coordinate decisions over spatially structured pickup and delivery regions. These characteristics make PDP a useful and challenging testbed for studying neural combinatorial optimization methods for constrained routing.

In this paper, we focus on a classical static single-vehicle PDP with one depot and n pickup–delivery pairs in a two-dimensional Euclidean space. The objective is to construct a Hamiltonian tour that starts and ends at the depot, visits each pickup and delivery node exactly once, satisfies all pickup-before-delivery constraints, and minimizes the total travel distance. This setting abstracts away application-specific constraints such as time windows, capacities, heterogeneous fleets, and dynamic arrivals, allowing us to isolate a fundamental architectural question: how should a neural construction policy represent and decode the spatial and precedence structures inherent in PDP?

Traditional exact and heuristic methods provide important baselines for PDP and related routing problems [7, 27]. Exact formulations can certify optimality on small or medium instances, but their computational cost grows rapidly with the number of requests and operational constraints. Heuristic and metaheuristic methods, such as insertion heuristics and adaptive large neighborhood search, scale better in practice but often depend on hand-crafted neighborhoods, problem-specific repair rules, and careful parameter tuning. This dependence motivates learning-based solvers that can automatically acquire routing policies from data and reuse them across many problem instances.

Deep reinforcement learning (DRL) and neural combinatorial optimization have made substantial progress in this direction. Pointer-network and attention-based encoder–decoder models can construct tours directly from node coordinates [3, 13, 25, 30], while POMO-style training improves sample efficiency by exploiting multiple symmetric rollouts per instance [14]. In parallel, neural improvement, learned local

search, and diffusion-based solvers further improve solution quality through refinement or generative sampling procedures [4, 28, 32]. For PDP, recent methods incorporate pickup-delivery structure through role-aware attention, precedence-aware masking, or learned neighborhood search. Representative examples include Heter-AM, which distinguishes depot, pickup, and delivery nodes within an attention-based policy [17], ENNS, which learns PDP-specific neighborhood search operators [20], and NCS, which couples neural construction with collaborative ruin-and-repair improvement [12].

However, these advances leave an important architectural gap. Many neural PDP solvers still operate primarily on a flat node graph: all nodes interact through generic attention, while region-level spatial structure is expected to emerge implicitly during training. This is restrictive for PDP instances whose pickup and delivery nodes exhibit coherent spatial organization. In such cases, high-quality routes require a multi-scale decision pattern: the policy should exploit local continuity within a region while also deciding when to transition between regions. A flat representation can capture this behavior only indirectly, and a single decoder must simultaneously handle local routing and global relocation decisions.

A second limitation concerns the trade-off between solution quality and inference cost. Search-augmented neural methods can achieve strong performance by repeatedly improving an initial solution, but this iterative refinement increases inference latency. In time-sensitive routing scenarios, a solver that requires many improvement iterations may be less attractive than a construction policy that produces high-quality feasible solutions through autoregressive decoding. Thus, the limitation is not simply that current neural PDP solvers are inaccurate, but that their architectures provide weak inductive bias for the multi-scale structure of PDP instances. A desirable neural PDP policy should preserve global feasibility, capture local region-level geometry, distinguish local routing from global transitions, and reduce reliance on expensive iterative refinement at inference time.

These observations motivate Cluster-Aware Attention-based Deep Reinforcement Learning (CAADRL), a cluster-aware neural construction framework for PDP. The core idea is to make the multi-scale structure of PDP explicit at both the representation and decoding levels. At the representation level, CAADRL combines global self-attention with intra-cluster attention so that node embeddings encode both instance-wide geometry and local role-specific structure. At the decision level, CAADRL uses a hierarchical Dynamic Dual-Decoder to separately model intra-cluster routing and inter-cluster transitions. The two decoding branches are combined at the action-distribution level, allowing the policy to integrate local routing preferences and global transition preferences while maintaining autoregressive construction. Feasibility is enforced through precedence-aware masking, and the whole policy is trained end-to-end using a POMO-style policy gradient scheme.

The main contributions of this work are summarized as follows:

- **Cluster-aware representation.** We propose a cluster-aware Transformer encoder that combines global self-attention with intra-cluster attention, enabling PDP policies to explicitly capture depot, pickup, and delivery region structures.

Under the Sampling-1280 decoding setting, CAADRL reduces the paired objective over the Heterogeneous Attention baseline by 27.37% on PDP40-clustered and 47.42% on PDP80-clustered.

- **Hierarchical autoregressive decoding.** We design a Dynamic Dual-Decoder that separately models intra-cluster continuation and inter-cluster transition preferences, and combines them into a feasible action distribution. On PDP80-uniform, CAADRL with Sampling-12800 achieves a 7.49% lower paired objective than NCS with 3000 collaborative-search iterations, while avoiding iterative ruin-and-repair refinement.
- **Comprehensive evaluation.** We conduct 10,000-instance experiments with paired statistical tests, ablation studies, route-level diagnostics, and runtime and memory profiling. The results show that CAADRL achieves strong solution quality with sub-0.20-second per-instance inference on PDP80 in the main benchmark setting.

The remainder of this paper is organized as follows. Section 2 reviews exact, heuristic, and learning-based methods for PDP and related routing problems. Section 3 introduces the formal problem definition and the feasibility constraints used in autoregressive decoding. Section 4 presents the proposed cluster-aware encoder, hierarchical dual-decoder, and POMO-based training procedure. Section 5 reports the experimental results, including baseline comparisons, ablation analysis, statistical testing, mechanism diagnostics, and efficiency profiling. Section 6 concludes the paper and discusses limitations and future directions.

2 Related Work

This section reviews three lines of work related to PDP: classical optimization methods, neural routing methods, and learning-based PDP solvers. We focus on how these methods differ in scalability, inference cost, and their ability to represent structured pickup–delivery relations.

2.1 Classical Optimization Methods for PDP

The PDP extends classical vehicle routing models by coupling pickup and delivery requests with precedence constraints. Surveys summarize its major variants, exact formulations, and applications in shared mobility and on-demand logistics [23, 27]. Exact methods based on mixed-integer programming, branch-and-cut, branch-and-price, and branch-and-cut-and-price provide rigorous benchmarks for small and medium instances. Recent studies further improve exact PDP algorithms through neighborhood structures, exponential-size neighborhoods, valid inequalities, and decompositions for richer variants such as multi-agent and multicompartment PDPs [1, 6, 11, 15, 26].

Despite their theoretical value, exact methods scale poorly as the number of requests and side constraints increases. Heuristic and metaheuristic methods are therefore widely used for larger PDP instances. Insertion heuristics and large neighborhood search construct and improve feasible routes through carefully designed local moves, destroy and repair operations, and penalty mechanisms. ALNS and its variants have

been applied to PDPs with time windows, scheduled lines, split loads, and transshipment structures [7, 18]. However, their performance often depends on hand-crafted neighborhoods and problem-specific tuning. This limits portability across PDP variants and motivates learning-based solvers that can acquire reusable routing policies from data.

2.2 Neural Routing and Search Methods

Neural combinatorial optimization learns routing policies directly from problem instances. Pointer networks and attention-based encoder-decoder models have achieved strong results on TSP, CVRP, and related routing problems [3, 13, 25, 30]. POMO further improves policy-gradient training by exploiting multiple symmetric rollouts for each instance [14]. These construction policies are computationally attractive because they generate solutions in one autoregressive pass. However, most of them treat the input as a flat set of nodes, forcing higher-level spatial structures—such as coherent pickup and delivery regions—to be inferred implicitly from attention weights rather than represented explicitly. Although explicitly modeling cluster-level organization has shown promise in related domains like clustered VRP [33], existing methods do not accommodate the complex pairing and precedence constraints inherent in pickup-delivery routing.

Neural search and improvement methods follow a different paradigm. NeuRewriter, learned improvement heuristics, neural large neighborhood search, flexible neural k -opt, latent search-space methods, and diffusion-based solvers improve routes through rewrite operations, learned neighborhoods, exchange moves, latent exploration, or sampling [4, 8, 9, 21, 28, 32]. Hybrid methods also combine neural policies with classical local search [35]. These methods often obtain better final objectives than pure construction policies, but their inference cost grows with the number of refinement steps, sampled solutions, or search iterations. This creates a quality-latency trade-off, especially when fast routing decisions are required.

2.3 Learning-Based Solvers for PDP

Compared with TSP and CVRP, PDP requires neural solvers to handle pickup-delivery pairing, node roles, and precedence feasibility. Heter-AM introduces heterogeneous attention and precedence-aware masking to distinguish depot, pickup, and delivery nodes [17]. This role-aware design improves over generic attention models because it prevents pickups and deliveries from being treated identically. Nevertheless, Heter-AM mainly models node-type heterogeneity. It does not explicitly represent multi-scale spatial regions or separately model intra-cluster continuation and inter-cluster transition preferences. Therefore, clustered pickup-delivery structure still has to be recovered indirectly from a largely flat attention representation.

Search-based PDP solvers improve solution quality through iterative refinement. ENNS learns PDP-specific neighborhood operators and a learned selection policy [20], while NCS combines neural construction with collaborative ruin-and-repair improvement [12]. These methods are strong quality-oriented baselines because they can revise

an initial route after construction. However, their performance depends on inference-time search, so larger search budgets lead to higher latency. Moreover, they improve a route after it has been constructed, rather than embedding multi-scale pickup–delivery structure directly into a one-pass policy.

Recent DRL studies also consider dynamic, flexible, and backhaul-rich pickup and delivery settings [29, 31, 34]. These works demonstrate the importance of learning-based methods for practical PDP variants, but they mainly focus on richer operational settings rather than the architectural representation of clustered spatial structure. In contrast, our approach embeds cluster-aware spatial structure directly into the construction policy, allowing the model to exploit pickup–delivery regions during decoding rather than relying on post-hoc iterative improvement. By combining global and intra-cluster attention with hierarchical decoding, CAADRL improves structural expressiveness while preserving one-pass, low-latency inference.

3 Preliminary

We consider the classical single-vehicle Pickup and Delivery Problem (PDP) in a two-dimensional Euclidean space. Given a depot node x_0 and n transportation requests, each request consists of a pickup node and a corresponding delivery node that must be served by the same vehicle. Let

$$\mathbf{P} = \{x_i\}_{i=1}^n, \quad \mathbf{D} = \{x_{i+n}\}_{i=1}^n$$

denote the pickup and delivery node sets, respectively. Each pair (x_i, x_{i+n}) is logically coupled, and the delivery node x_{i+n} can only be visited after its corresponding pickup node x_i . The complete node set is therefore

$$\mathbf{X} = \{x_0\} \cup \mathbf{P} \cup \mathbf{D}.$$

A feasible PDP route starts from the depot, visits every pickup and delivery node exactly once, satisfies all pickup-before-delivery precedence constraints, and finally returns to the depot.

The objective is to minimize the total travel distance of the closed route. Since the vehicle speed is assumed to be constant, minimizing total distance is equivalent to minimizing total travel time.

For completeness, we first give a compact mathematical formulation of the PDP. Let D_{ij} be the Euclidean distance between nodes x_i and x_j , and let f denote the constant vehicle speed. The binary variable $y_{ij} \in \{0, 1\}$ equals 1 if the route travels directly from node i to node j , and 0 otherwise. Let B_i denote the arrival time, or equivalently the cumulative travel time, when node i is reached. We set $B_0 = 0$ as the departure time from the depot. The PDP can be formulated as follows:

$$\min \sum_{i \in \mathbf{X}} \sum_{\substack{j \in \mathbf{X} \\ j \neq i}} \frac{D_{ij}}{f} y_{ij}. \quad (1)$$

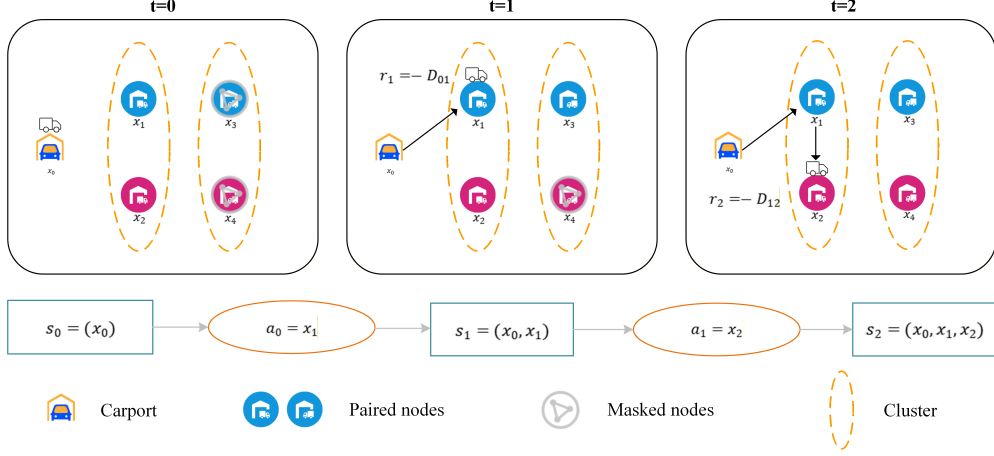


Fig. 1 Illustration of PDP route construction as a sequential decision-making process. At each step, the policy selects one feasible unvisited node and appends it to the partial route. Nodes that have already been visited or whose precedence requirements are not yet satisfied are masked out. The dashed orange ovals indicate the pickup and delivery regions used as cluster-level structural information in our model. This cluster information guides representation learning and hierarchical route construction, while feasibility is enforced by the dynamic action mask.

subject to

$$\sum_{\substack{j \in \mathbf{X} \\ j \neq i}} y_{ij} = 1, \quad \forall i \in \mathbf{X}, \quad (2)$$

$$\sum_{\substack{i \in \mathbf{X} \\ i \neq j}} y_{ij} = 1, \quad \forall j \in \mathbf{X}, \quad (3)$$

$$B_0 = 0, \quad (4)$$

$$B_j \geq B_i + \frac{D_{ij}}{f} - M(1 - y_{ij}), \quad \forall i \in \mathbf{X}, \forall j \in \mathbf{X} \setminus \{x_0\}, i \neq j, \quad (5)$$

$$B_{i+n} \geq B_i, \quad \forall i = 1, \dots, n. \quad (6)$$

Constraints (2) and (3) ensure that each node has exactly one outgoing and one incoming edge in the final tour. Constraint (4) fixes the departure time at the depot. Constraint (5) imposes a consistent visiting order along selected edges: when $y_{ij} = 1$, the arrival time at node j must be no earlier than the arrival time at node i plus the corresponding travel time. The big- M term deactivates this implication when edge (i, j) is not selected. The return edge to the depot is excluded from this sequencing constraint to avoid imposing a cyclic arrival-time order on the closed tour, while the closed route itself is still enforced by the degree constraints. Constraint (6) encodes the essential pickup-before-delivery rule for each request pair.

It is important to note that the above mixed-integer program is not directly solved during either training or inference. Instead, it serves as a formal description

of the objective and feasible solution space. In our learning-based solver, a route is constructed autoregressively. At decoding step t , the state is the partial route

$$s_t = (x_0, \tau_1, \dots, \tau_t),$$

where $\tau_1, \dots, \tau_t$ are the customer nodes already selected by the policy. Let

$$\mathcal{V}_t = \{\tau_1, \dots, \tau_t\}$$

denote the visited customer-node set. The action is the next node to be appended to the partial route. Feasibility is enforced by a dynamic action mask rather than by solving the MIP constraints. Specifically, an unvisited pickup node is feasible, while an unvisited delivery node x_{i+n} is feasible only if its paired pickup node x_i has already appeared in the partial route. For $t < 2n$, the feasible action set is

$$\mathcal{A}(s_t) = \{x_i \in \mathbf{P} : x_i \notin \mathcal{V}_t\} \cup \{x_{i+n} \in \mathbf{D} : x_{i+n} \notin \mathcal{V}_t, x_i \in \mathcal{V}_t\}. \quad (7)$$

After all pickup and delivery nodes have been selected, the route returns to the depot x_0 deterministically, and the return edge is included in the final route length. During decoding, all nodes outside $\mathcal{A}(s_t)$ are assigned zero probability by adding $-\infty$ to their logits before the softmax operation. In this way, the visit-once requirement in constraints (2)–(3) is satisfied by selecting each customer node once, and the precedence constraint (6) is enforced by the dynamic feasibility mask. The sequencing variables B_i are therefore not optimized in the neural solver; the visiting order is directly determined by the autoregressively generated route.

The cluster structure considered in this paper is incorporated differently from the hard PDP constraints above. It is not an additional feasibility constraint and does not change the feasible solution space. Instead, each node is associated with a role-based cluster label,

$$c_i \in \{0, 1, 2\},$$

where $c_i = 0$ denotes the depot, $c_i = 1$ denotes a pickup node, and $c_i = 2$ denotes a delivery node. These labels define the depot, pickup, and delivery groups used by the cluster-aware encoder and the hierarchical decoder. In clustered instances, these role-based groups are aligned with spatially separated pickup and delivery regions; in uniform instances, they still provide pickup–delivery role information without imposing any geometric separation. Thus, the mathematical constraints define what routes are feasible, the dynamic mask enforces these constraints during decoding, and the cluster representation provides an architectural prior that helps the policy learn when to exploit local intra-region structure and when to perform inter-region transitions.

4 Methodology

This section presents CAADRL for solving PDP as a masked autoregressive reinforcement learning problem. We first formulate route construction as a sequential decision process, then introduce the policy architecture composed of a cluster-aware

Transformer encoder and a hierarchical dual-branch decoder. The encoder captures both global spatial dependencies and role-based intra-cluster structure, while the decoder separately models intra-cluster continuation and inter-cluster transition preferences. Each decoder branch first produces a masked probability distribution over feasible nodes. The two distributions are then combined by fixed probability-level fusion, followed by feasibility masking and renormalization. This ensures that the final policy assigns positive probability only to feasible actions. We finally analyze the computational profile and describe the POMO-based training scheme.

4.1 RL Formulation

PDP route construction can be modeled as a sequential decision-making process [3, 13, 25]. As shown in Fig. 1, a partial route is extended step by step by selecting one feasible unvisited node at each decoding step.

State. At step t , the state is the partial route

$$s_t = (\tau_0, \tau_1, \dots, \tau_t),$$

where $\tau_0 = x_0$ is the depot and $\tau_1, \dots, \tau_t$ are the customer nodes already selected. The state determines both the current vehicle location and the visited-node set.

Action. The action A_t is the next node to append to the route. The feasible action set $\mathcal{A}(s_t)$ contains unvisited pickup nodes and unvisited delivery nodes whose paired pickups have already been visited. This feasibility rule is implemented by the dynamic mask defined in Section 3.

Transition. After selecting $A_t = \tau_{t+1}$, the route is updated deterministically:

$$s_{t+1} = (\tau_0, \tau_1, \dots, \tau_t, \tau_{t+1}).$$

Reward. The incremental reward is the negative travel distance:

$$r_{t+1} = r(s_t, A_t) = -D_{\tau_t, \tau_{t+1}}. \quad (8)$$

After all $2n$ customer nodes are visited, the vehicle returns to the depot deterministically. The total reward of route $\tau = (\tau_0, \tau_1, \dots, \tau_{2n})$ is

$$R(\tau) = - \left(\sum_{t=0}^{2n-1} D_{\tau_t, \tau_{t+1}} + D_{\tau_{2n}, x_0} \right). \quad (9)$$

Maximizing $R(\tau)$ is equivalent to minimizing the closed-route length.

Policy. The stochastic policy π_θ generates a feasible customer-node permutation autoregressively:

$$P(\tau|\mathbf{X}; \theta) = \prod_{t=0}^{2n-1} \pi_\theta(\tau_{t+1}|\mathbf{X}, \tau_{0:t}), \quad (10)$$

where the final return to the depot is deterministic and included only in the route cost. The overall policy network is summarized in Fig. 2.

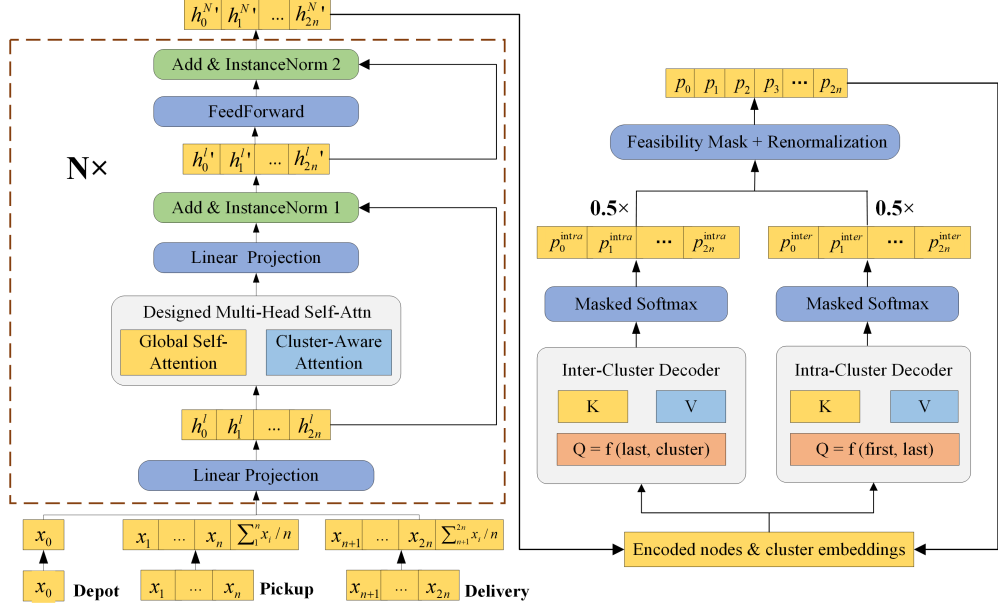


Fig. 2 Cluster-aware policy network. The encoder combines global self-attention and intra-cluster attention. The decoder produces intra-cluster and inter-cluster masked probability distributions in parallel, then combines them through probability-level fusion followed by feasibility masking and renormalization.

4.2 Policy Network Architecture

We parameterize π_θ with a Transformer-based encoder–decoder model [13]. The design is motivated by the fact that PDP requires both global route-level reasoning and local role-specific reasoning. Global attention captures instance-wide geometry, while role-based intra-cluster attention emphasizes the depot, pickup, and delivery structures.

4.2.1 Encoder with Cluster-Aware Attention

Each node x_i is represented by its 2D coordinates and a role-based cluster label $c_i \in \{0, 1, 2\}$, where 0, 1, and 2 denote the depot, pickup nodes, and delivery nodes, respectively. The input feature is projected into a d_h -dimensional embedding:

$$h_i^{(0)} = W_{\text{in}}[x_i, c_i] + b_{\text{in}}. \quad (11)$$

Before the main encoder stack, a depot-centered cross-attention operation updates the depot representation:

$$h_0^{(0)} \leftarrow \text{MHA} \left(h_0^{(0)}, H_{\text{cust}}^{(0)}, H_{\text{cust}}^{(0)} \right), \quad (12)$$

where $H_{\text{cust}}^{(0)}$ contains all pickup and delivery embeddings.

The main encoder contains L cluster-aware attention layers. Given $H^{(l-1)} \in \mathbb{R}^{|\mathbf{X}| \times d_h}$, each layer computes a global attention branch and an intra-cluster attention branch. The global branch is standard multi-head self-attention:

$$Q_G = H^{(l-1)}W_Q^G, \quad K_G = H^{(l-1)}W_K^G, \quad V_G = H^{(l-1)}W_V^G, \quad (13)$$

$$H_G = \text{Concat}(\text{head}_1^G, \dots, \text{head}_h^G)W_O^G, \quad (14)$$

where

$$\text{head}_r^G = \text{softmax} \left(\frac{Q_G^{(r)}(K_G^{(r)})^\top}{\sqrt{d_k}} \right) V_G^{(r)}. \quad (15)$$

The intra-cluster branch uses a structural mask

$$M_{ij}^{\text{cluster}} = \begin{cases} 0, & c_i = c_j, \\ -\infty, & c_i \neq c_j, \end{cases} \quad (16)$$

which restricts attention to nodes with the same role-based cluster label. With independent projections

$$Q_C = H^{(l-1)}W_Q^C, \quad K_C = H^{(l-1)}W_K^C, \quad V_C = H^{(l-1)}W_V^C, \quad (17)$$

the masked attention output is

$$H_C = \text{Concat}(\text{head}_1^C, \dots, \text{head}_h^C)W_O^C, \quad (18)$$

where

$$\text{head}_r^C = \text{softmax} \left(\frac{Q_C^{(r)}(K_C^{(r)})^\top}{\sqrt{d_k}} + M^{\text{cluster}} \right) V_C^{(r)}. \quad (19)$$

The two branches are fused through a residual Transformer block:

$$\tilde{H}^{(l)} = \text{LN} \left(H^{(l-1)} + W_F(H_G + H_C) \right), \quad (20)$$

$$H^{(l)} = \text{LN} \left(\tilde{H}^{(l)} + \text{FFN}(\tilde{H}^{(l)}) \right). \quad (21)$$

After L layers, the final node embeddings are

$$H = [h_0, h_1, \dots, h_{2n}]^\top.$$

4.2.2 Hierarchical Dual-Branch Decoder

Given H , the decoder constructs two probability distributions at each decision step: an intra-cluster distribution and an inter-cluster distribution. These two branches have different query definitions but share the same dynamic feasibility mask.

Let $h_{\text{last}} = h_{\tau_t}$ be the embedding of the current node. For $t \geq 1$, let $h_{\text{first}} = h_{\tau_1}$ be the embedding of the first customer node already visited in the current partial

tour; before any customer node has been selected, the depot embedding is used as a placeholder. Let $\bar{h}_{c_t}$ be the mean embedding of nodes with the same role-based cluster label as the current node, where $c_t = c_{\tau_t}$. The intra-cluster and inter-cluster queries are

$$q_{\text{intra}} = W_q^{\text{first}} h_{\text{first}} + W_q^{\text{last}} h_{\text{last}}, \quad (22)$$

and

$$q_{\text{inter}} = \widetilde{W}_q^{\text{last}} h_{\text{last}} + W_q^{\text{cluster}} \bar{h}_{c_t}. \quad (23)$$

For $b \in \{\text{intra}, \text{inter}\}$, let q_b denote one branch query. The decoder computes

$$K = HW_K, \quad V = HW_V. \quad (24)$$

The dynamic feasibility mask is

$$(M_t^{\text{mask}})_i = \begin{cases} 0, & x_i \in \mathcal{A}(s_t), \\ -\infty, & x_i \notin \mathcal{A}(s_t). \end{cases} \quad (25)$$

Each branch first computes a masked attention glimpse:

$$a_b^t = \text{softmax} \left(\frac{q_b K^\top}{\sqrt{d_k}} + M_t^{\text{mask}} \right), \quad (26)$$

$$g_b^t = a_b^t V. \quad (27)$$

Treating g_b^t and k_i as row vectors, the branch score for node x_i is

$$u_{b,i}^t = \frac{g_b^t(k_i)^\top}{\sqrt{d_k}} = \frac{\langle g_b^t, k_i \rangle}{\sqrt{d_k}}, \quad (28)$$

and the clipped masked score is

$$\hat{u}_{b,i}^t = C \tanh(u_{b,i}^t) + (M_t^{\text{mask}})_i. \quad (29)$$

The branch output is a masked probability distribution:

$$P_{b,i}^t = \frac{\exp(\hat{u}_{b,i}^t)}{\sum_{j=0}^{2n} \exp(\hat{u}_{b,j}^t)}, \quad b \in \{\text{intra}, \text{inter}\}. \quad (30)$$

The final policy is obtained by fixed probability-level fusion:

$$\tilde{P}_i^t = \frac{1}{2} P_{\text{intra},i}^t + \frac{1}{2} P_{\text{inter},i}^t. \quad (31)$$

Although branch probabilities are theoretically zero for infeasible nodes, we apply explicit masking and renormalization in Eq. (32) for numerical robustness:

$$\pi_\theta(x_i | \mathbf{X}, \tau_{0:t}) = \frac{\mathbf{1}[x_i \in \mathcal{A}(s_t)] \tilde{P}_i^t}{\sum_{j=0}^{2n} \mathbf{1}[x_j \in \mathcal{A}(s_t)] \tilde{P}_j^t}. \quad (32)$$

At inference time, we use greedy decoding or sampling-based decoding. Sampling generates multiple candidate routes and returns the best one, but each candidate is still produced by masked autoregressive construction without iterative ruin-and-repair refinement.

4.3 Complexity and Architectural Rationale

Let $m = |\mathbf{X}| = 2n + 1$. The global attention branch has complexity $O(m^2 d_h)$ per encoder layer. The intra-cluster branch restricts effective interactions to nodes with the same role-based cluster label. In PDP, the role-based clusters have sizes 1, n , and n , so the effective intra-cluster interaction pattern contains $1 + 2n^2$ node pairs. In our dense masked implementation, the full attention matrix is still computed, so the wall-clock complexity remains $O(m^2 d_h)$. Therefore, intra-cluster attention is used as a structured inductive bias rather than as an asymptotic speed-up.

The decoder has two masked probability branches. Each branch costs $O(m d_h)$ per step, and a complete autoregressive construction costs $O(m^2 d_h)$ per candidate route. Thus, CAADRL keeps the same quadratic asymptotic order as standard attention-based construction policies, with a moderate constant-factor overhead from the additional encoder and decoder branches.

Compared with Heter-AM-like construction models, CAADRL remains in the same autoregressive complexity class but adds explicit role-based intra-cluster modeling and separate intra/inter-cluster decoding preferences. Compared with NCS-like collaborative improvement methods, CAADRL does not apply iterative ruin-and-repair refinement after route construction. If such a method uses K refinement iterations, its inference cost contains an additional iteration-dependent factor. By contrast, each CAADRL candidate route is generated by one masked construction pass; when sampling is used, cost grows linearly with the number of sampled candidates.

The dynamic mask guarantees feasibility during decoding, while logit clipping and the POMO baseline provide practical training stabilization rather than a formal convergence guarantee.

4.4 Training with POMO

We train the policy using REINFORCE with a shared instance-wise baseline and adapt the Policy Optimization with Multiple Optima (POMO) framework [14]. For a batch of B instances $\{X^b\}_{b=1}^B$, we generate N feasible rollouts for each instance. In PDP, the initial node must be a feasible pickup node. For instance X^b , the rollouts are denoted by $\{\tau^{b,j}\}_{j=1}^N$ with rewards $\{R^{b,j}\}_{j=1}^N$.

The shared baseline is

$$b(X^b) = \frac{1}{N} \sum_{j=1}^N R^{b,j}, \quad (33)$$

and the rollout advantage is

$$A^{b,j} = R^{b,j} - b(X^b). \quad (34)$$

The policy-gradient estimator is

$$\nabla_{\theta} J(\theta) \approx \frac{1}{B} \sum_{b=1}^B \frac{1}{N} \sum_{j=1}^N A^{b,j} \nabla_{\theta} \log P(\tau^{b,j} | X^b; \theta), \quad (35)$$

where

$$\log P(\tau^{b,j} | X^b; \theta) = \sum_{t=0}^{2n-1} \log \pi_{\theta}(\tau_{t+1}^{b,j} | X^b, \tau_{0:t}^{b,j}).$$

Equivalently, the training loss is

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{b=1}^B \frac{1}{N} \sum_{j=1}^N A^{b,j} \log P(\tau^{b,j} | X^b; \theta). \quad (36)$$

Algorithm 1 POMO-based Training for CAADRL

```

1: Input: epochs  $I$ , batch size  $B$ , rollouts  $N$ , policy  $\pi_{\theta}$ , learning rate  $\eta$ 
2: for epoch = 1 to  $I$  do
3:   Generate a batch of PDP instances  $\{X^b\}_{b=1}^B$ 
4:   for each instance  $X^b$  do
5:     for  $j = 1$  to  $N$  do
6:       Initialize rollout  $j$  from a feasible pickup node
7:       Construct route  $\tau^{b,j}$  using masked autoregressive decoding
8:       Compute reward  $R^{b,j}$  by Eq. (9)
9:     end for
10:    Compute  $b(X^b) = \frac{1}{N} \sum_{j=1}^N R^{b,j}$  and  $A^{b,j} = R^{b,j} - b(X^b)$ 
11:  end for
12:  Compute  $\mathcal{L}(\theta)$  by Eq. (36)
13:  Update  $\theta \leftarrow \text{Adam}(\theta, \nabla_{\theta} \mathcal{L}(\theta), \eta)$ 
14: end for

```

All rollouts remain feasible because the same precedence-aware dynamic mask is used throughout decoding. The shared baseline reduces variance without an additional critic network, and multiple rollouts provide diverse learning signals for each instance.

5 Computational Experimentation and Analysis

This section evaluates CAADRL from five complementary perspectives: solution quality, statistical reliability, architectural contribution, route-level behavior, and inference efficiency. We first compare CAADRL with representative neural PDP solvers on clustered and uniform synthetic benchmarks. We then report paired instance-level statistics, ablation results, route diagnostics, runtime and memory profiling, an external Li&Lim relaxed evaluation, and large-scale size transfer.

5.1 Experimental Settings

Following common practice in neural combinatorial optimization [13, 14], synthetic instances are generated on the unit square $[0, 1] \times [0, 1]$, and Euclidean distance is used as the travel cost. We consider two distributions. In the **clustered distribution**, pickup nodes are sampled from a normal distribution centered at $(0.25, 0.25)$, and delivery nodes are sampled from another normal distribution centered at $(0.75, 0.75)$; the standard deviation is 0.1 for both distributions. This setting creates two spatially separated regions and directly tests whether the model can exploit local intra-region routing and global inter-region transitions. In the **uniform distribution**, depot, pickup, and delivery nodes are uniformly sampled from $[0, 1] \times [0, 1]$. This setting removes explicit spatial separation and tests whether the cluster-aware inductive bias remains useful when the preferred clustered structure is weak.

We evaluate PDP10, PDP20, PDP40, and PDP80, where the number denotes the total number of customer nodes. For example, PDP80 contains 40 pickup-delivery pairs. Unless otherwise stated, the embedding dimension is $d_h = 128$, the encoder has $L = 6$ layers, each attention module uses 8 heads, and the logit clipping constant is $C = 10$. Models are trained for 800 epochs with batch size 512 using Adam with learning rate 10^{-4} .

In all main experiments, CAADRL uses the architecture described in Section 4: a cluster-aware encoder and a hierarchical dual-branch decoder with fixed probability-level average fusion. Each decoder branch first produces a masked probability distribution, and the two distributions are averaged and re-normalized over feasible actions.

We compare CAADRL with two representative neural PDP solvers. **NCS** combines neural construction with iterative ruin-and-repair improvement [12]; we report $t = 1000$, $t = 2000$, and $t = 3000$ improvement iterations. **Heter** is a heterogeneous-attention encoder-decoder model that distinguishes depot, pickup, and delivery node roles [17]. For CAADRL and Heter, we evaluate Greedy, Sampling-1280, and Sampling-12800. Sampling-1280 and Sampling-12800 generate 1280 or 12800 complete routes from the learned policy and return the best route; they do not perform iterative ruin-and-repair refinement.

5.2 Training Convergence

Figs. 3–4 show the training curves under clustered and uniform generators. Across problem sizes, the objective decreases rapidly in the early training stage and then gradually stabilizes, indicating stable POMO-based policy-gradient optimization. Larger instances converge to higher objectives because they require longer routes and more precedence-constrained decisions. Clustered instances generally converge to lower objectives than uniform instances, reflecting that spatial regularity makes the routing structure more exploitable.

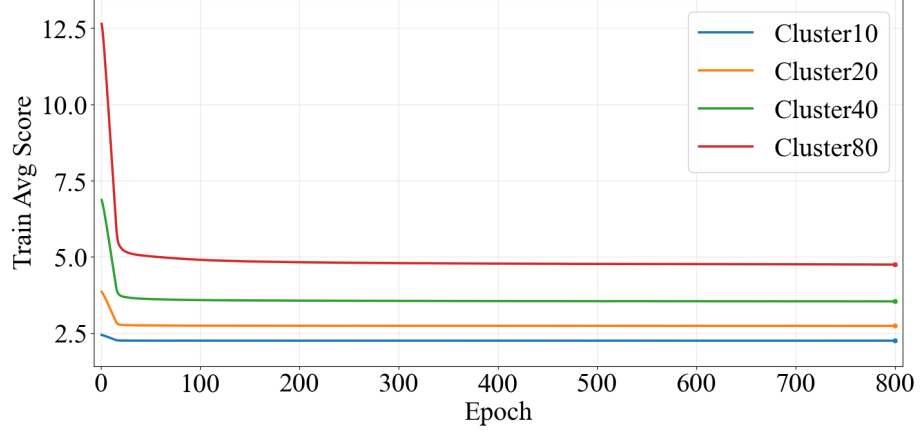


Fig. 3 Training convergence on clustered PDP instances. The curves report the training objective over epochs for PDP10/20/40/80 under the clustered generator.

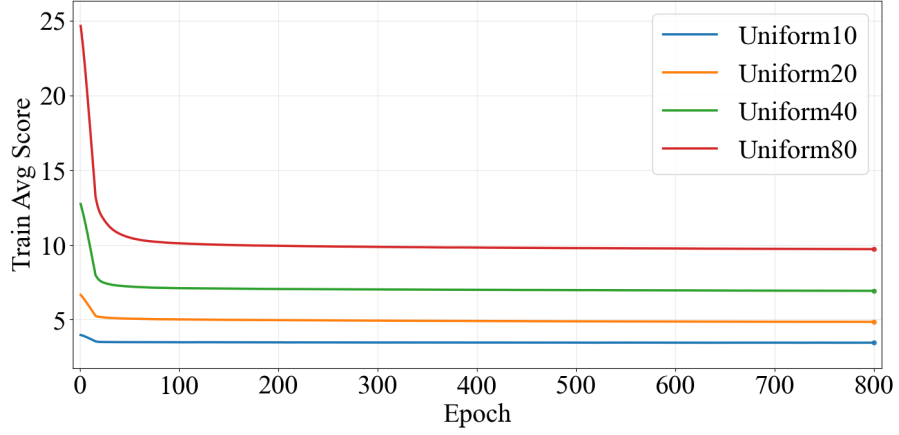


Fig. 4 Training convergence on uniform PDP instances. The curves report the training objective over epochs for PDP10/20/40/80 under the uniform generator.

5.3 Main Benchmark Results

Tables 1 and 2 report the final main benchmark comparison. For each problem size, Obj. is the mean route length, Gap is the relative gap to the best Obj. in the same column group, and Time is the per-instance inference time in seconds.

Tables 1 and 2 show two different regimes. On clustered instances, CAADRL becomes increasingly strong as the problem size grows. Heter with extensive sampling

Table 1 Performance comparison on clustered PDP instances. Lower Obj. and Gap are better.

Method	PDP10-clustered			PDP20-clustered			PDP40-clustered			PDP80-clustered		
	Obj.	Gap (%)	Time	Obj.	Gap (%)	Time	Obj.	Gap (%)	Time	Obj.	Gap (%)	Time
CAADRL-Greedy	2.258	0.97	0.024	2.745	0.18	0.041	3.566	0.75	0.073	4.815	1.98	0.138
CAADRL-S1280	2.258	0.95	0.027	2.740	0.02	0.046	3.542	0.07	0.084	4.732	0.23	0.159
CAADRL-S12800	2.258	0.95	0.027	2.740	0.00	0.046	3.539	0.00	0.086	4.721	0.00	0.179
NCS- $t = 1000$	2.259	1.02	0.102	2.747	0.25	0.117	3.692	4.32	0.144	10.733	127.33	0.214
NCS- $t = 2000$	2.253	0.75	0.205	2.743	0.12	0.239	3.660	3.41	0.285	10.394	120.17	0.431
NCS- $t = 3000$	2.250	0.63	0.312	2.742	0.07	0.351	3.644	2.94	0.424	10.244	116.97	0.626
Heter-Greedy	2.377	6.29	0.014	3.051	11.34	0.023	4.888	38.10	0.040	7.079	49.95	0.073
Heter-S1280	2.242	0.27	0.019	2.923	6.67	0.031	4.847	36.94	0.058	9.056	91.81	0.140
Heter-S12800	2.236	0.00	0.155	2.854	4.14	0.275	4.654	31.49	0.557	8.605	82.26	1.346

Table 2 Performance comparison on uniform PDP instances. Lower Obj. and Gap are better.

Method	PDP10-uniform			PDP20-uniform			PDP40-uniform			PDP80-uniform		
	Obj.	Gap (%)	Time	Obj.	Gap (%)	Time	Obj.	Gap (%)	Time	Obj.	Gap (%)	Time
CAADRL-Greedy	3.496	4.45	0.024	5.004	6.50	0.041	7.187	9.69	0.073	10.239	8.33	0.140
CAADRL-S1280	3.411	1.90	0.027	4.744	0.96	0.046	6.797	3.75	0.088	9.549	1.03	0.168
CAADRL-S12800	3.397	1.50	0.028	4.708	0.21	0.049	6.742	2.90	0.091	9.452	0.00	0.187
NCS- $t = 1000$	3.351	0.12	0.103	5.045	7.37	0.274	6.684	2.01	0.111	10.889	15.21	0.215
NCS- $t = 2000$	3.348	0.03	0.208	5.009	6.61	0.239	6.589	0.56	0.215	10.444	10.49	0.436
NCS- $t = 3000$	3.347	0.00	0.311	4.989	6.19	0.356	6.552	0.00	0.318	10.217	8.10	0.627
Heter-Greedy	3.474	3.79	0.014	5.115	8.88	0.023	7.508	14.59	0.040	10.961	15.96	0.074
Heter-S1280	3.371	0.70	0.018	4.741	0.90	0.031	7.143	9.02	0.059	10.512	11.21	0.138
Heter-S12800	3.362	0.44	0.149	4.698	0.00	0.277	7.086	8.14	0.549	10.410	10.13	1.351

is best on PDP10, but CAADRL achieves the best results on PDP20, PDP40, and PDP80. This pattern supports the central design hypothesis: when pickup and delivery nodes form coherent spatial regions, explicitly representing both global geometry and role-based local structure provides an increasingly useful inductive bias as the number of precedence-constrained decisions grows.

On uniform instances, CAADRL is not uniformly dominant, which is expected because the explicit pickup/delivery region structure is weaker. NCS is strongest on PDP10 and PDP40, while Heter is slightly better on PDP20. However, CAADRL obtains the best PDP80-uniform result. This suggests that the architecture is not merely specialized to the clustered generator: at larger sizes, even uniform PDP instances require a multi-scale policy that alternates many local moves with occasional longer transitions while maintaining precedence feasibility. Sampling improves CAADRL in the in-distribution benchmark; on PDP80-uniform, the objective decreases from 10.239 under Greedy to 9.549 under Sampling-1280 and 9.452 under Sampling-12800, without iterative route-improvement search.

5.4 Paired Statistical Analysis

Mean objectives alone do not show whether improvements are reliable at the instance level. We therefore report paired comparisons on aligned test outputs in Table 3. A negative paired gap means that CAADRL has a lower objective than the comparator

Table 3 Paired statistical comparison on clustered aligned test instances. Negative gaps favor CAADRL.

Size	Dist.	CAADRL	Comparator	Instances	Gap (%)	95% CI	p	W/T/L
PDP20	Clustered	Greedy	Heter-Greedy	10000	-9.71	[-9.79,-9.63]	< 0.001	9980/0/20
		S1280	Heter-S1280	10000	-6.13	[-6.18,-6.08]	< 0.001	9919/0/81
PDP40	Clustered	Greedy	Heter-Greedy	10000	-26.85	[-26.98,-26.72]	< 0.001	10000/0/0
		S1280	Heter-S1280	10000	-27.37	[-27.46,-27.28]	< 0.001	10000/0/0
PDP80	Clustered	Greedy	Heter-Greedy	10000	-30.23	[-30.39,-30.07]	< 0.001	10000/0/0
		S1280	Heter-S1280	10000	-47.42	[-47.49,-47.36]	< 0.001	10000/0/0
PDP20	Clustered	S1280	NCS- $t = 3000$	10000	-0.040	[-0.056,-0.024]	< 0.001	3690/3885/2425
		S12800	NCS- $t = 3000$	10000	-0.056	[-0.071,-0.040]	< 0.001	3845/3962/2193
PDP40	Clustered	S1280	NCS- $t = 3000$	10000	-2.749	[-2.791,-2.708]	< 0.001	9109/1/890
		S12800	NCS- $t = 3000$	10000	-2.816	[-2.858,-2.775]	< 0.001	9162/0/838
PDP80	Clustered	S1280	NCS- $t = 3000$	10000	-53.718	[-53.766,-53.671]	< 0.001	10000/0/0
		S12800	NCS- $t = 3000$	10000	-53.825	[-53.872,-53.777]	< 0.001	10000/0/0

Table 4 Core ablation study on clustered instances. Lower objective is better.

Variant	PDP20		PDP40		PDP80	
	Greedy	S1280	Greedy	S1280	Greedy	S1280
CAADRL	2.745	2.740	3.566	3.542	4.815	4.732
Global-only encoder	2.747	2.741	3.582	3.548	4.857	4.770
Cluster-only encoder	2.745	2.742	3.570	3.544	4.833	4.758
Single decoder	2.746	2.741	3.581	3.546	4.837	4.747
Single rollout, no POMO	4.621	4.885	8.214	11.205	15.286	24.710

on average. The table also reports confidence intervals and p -values computed from paired differences.

Table 3 shows that the improvements are not merely a table-level averaging effect. Against Heter, CAADRL wins essentially all aligned clustered instances at PDP40 and PDP80 under the reported decoding settings. Against NCS, CAADRL is slightly worse or statistically tied at very small clustered sizes, but the advantage becomes clear at larger clustered sizes. This is particularly relevant because NCS uses 3000 improvement iterations, whereas CAADRL returns each sampled candidate through one masked autoregressive construction pass.

5.5 Ablation Study

We next isolate the contribution of the encoder, decoder, fusion design, and training scheme. The main CAADRL row uses the fixed probability-level average-fusion dual-branch decoder described in Section 4. We compare it with a global-only encoder, a cluster-only encoder, a single-decoder variant, and a single-rollout training variant without POMO-style multiple rollouts.

Table 4 shows that POMO-style multiple-rollout training is essential: removing it leads to severe degradation across all sizes. Among architectural variants, the differences are small on PDP20 but become clearer on PDP40 and PDP80. The global-only

Table 5 Route-level diagnostics for CAADRL. Higher intra-edge ratio and separability indicate stronger role-region continuity.

Dataset	Decode	Instances	Switches	Intra-edge ratio	Separability
PDP20-clustered	Greedy	10000	1.000	0.947	5.411
PDP20-clustered	S12800	10000	1.000	0.947	5.411
PDP40-clustered	Greedy	10000	1.000	0.974	5.199
PDP40-clustered	S12800	10000	1.000	0.974	5.199
PDP80-clustered	Greedy	10000	1.000	0.987	5.117
PDP80-clustered	S12800	10000	1.000	0.987	5.117
PDP20-uniform	Greedy	10000	6.328	0.667	0.427
PDP20-uniform	S12800	10000	6.729	0.646	0.427
PDP40-uniform	Greedy	10000	12.028	0.692	0.290
PDP40-uniform	S12800	10000	12.343	0.684	0.290
PDP80-uniform	Greedy	10000	27.353	0.654	0.200
PDP80-uniform	S12800	10000	26.947	0.659	0.200

and cluster-only rows show that neither instance-wide attention nor role-local attention alone is sufficient at larger scales; the two views are most useful when combined. The single-decoder row remains competitive, but the full average-fusion dual decoder is strongest on the larger clustered instances, indicating that fixed probability-level averaging is sufficient to combine local-routing and global-transition preferences.

5.6 Route-Level Diagnostics

To analyze whether CAADRL uses the intended route structure, we compute three route-level diagnostic metrics. *Role-region switches* count how many times two consecutive customer visits in the final route belong to different role-based regions, such as moving from a pickup node to a delivery node. A smaller switch count therefore indicates that the route tends to complete one role-region before moving to another. The *intra-cluster edge ratio* is the fraction of consecutive route edges whose two end-points have the same role-based cluster label; higher values mean that the policy preserves local continuity within pickup or delivery regions. The *cluster separability index* summarizes the geometric separation between the pickup and delivery point clouds, with larger values indicating that the two regions are more clearly separated in the input instance. Together, these metrics distinguish whether good objective values come from the intended region-aware routing behavior or from an unstructured sequence of feasible moves.

Table 5 explains why the architecture is especially strong on large clustered instances. In the clustered generator, pickup and delivery regions are well separated, and CAADRL produces routes with one dominant transition between regions and very high intra-region continuity. On PDP80-clustered, the intra-cluster edge ratio reaches 0.987. On uniform instances, separability is much lower and switch counts are higher, so the model cannot rely on the same simple regional structure. The behavior is therefore distribution-dependent rather than a fixed route template.

Table 6 PDP80 runtime and memory summary. Latency is seconds per instance; peak GPU memory is allocated memory.

Dist.	Method	Decode	Latency	Throughput	Peak GPU MB
Clustered	CAADRL	Greedy	0.151	6.63	22.37
	CAADRL	S1280	0.175	5.70	32.49
	CAADRL	S12800	0.191	5.24	186.40
	Heter	Greedy	0.073	13.62	14.46
	Heter	S1280	0.140	7.12	369.50
	Heter	S12800	1.346	0.74	408.70
	NCS	$t = 1000$	0.214	4.67	510.00
	NCS	$t = 2000$	0.431	2.32	510.00
	NCS	$t = 3000$	0.626	1.60	512.00
Uniform	CAADRL	Greedy	0.153	6.56	24.34
	CAADRL	S1280	0.197	5.08	32.49
	CAADRL	S12800	0.195	5.12	186.40
	Heter	Greedy	0.074	13.54	14.46
	Heter	S1280	0.138	7.24	369.50
	Heter	S12800	1.351	0.74	408.70
	NCS	$t = 1000$	0.215	4.66	510.00
	NCS	$t = 2000$	0.436	2.29	510.00
	NCS	$t = 3000$	0.627	1.59	512.00

5.7 Runtime and Memory Profiling

The Time columns in Tables 1 and 2 already provide per-instance inference costs in the main comparison, including the iteration-dependent NCS search budgets. To make the PDP80 efficiency comparison clearer, Table 6 consolidates latency, throughput, and peak GPU memory.

Table 6 supports the computational interpretation in Section 4. CAADRL remains an autoregressive construction policy with moderate constant-factor overhead from cluster-aware attention and dual-branch decoding. Even under Sampling-12800, PDP80 latency remains below 0.20 seconds per instance in the profiling run, and peak allocated GPU memory remains below 190 MB. Heter is faster under greedy construction, but its Sampling-12800 rows are substantially slower and more memory-intensive on PDP80. For NCS, it increases monotonically from $t = 1000$ to $t = 3000$. This reflects the different computational paradigm of NCS: route quality is improved through iterative search rather than one-pass construction.

5.8 External Li&Lim-PDP-Relaxed Evaluation

To reduce dependence on purely synthetic unit-square coordinates, we additionally evaluate on relaxed Li&Lim pickup-and-delivery instances [16]. The original Li&Lim instances are PDPTW benchmarks: they include time windows, capacity constraints, multiple vehicles, and objectives defined for the full PDPTW setting. This paper, however, studies the static single-vehicle distance-only PDP. We therefore retain the

Table 7 External evaluation on Li&Lim relaxed distance-only PDP instances.

Method	Source group	Decode	Instances	Mean length	Std.	Avg. time (s)
CAADRL	100	Greedy	56	2961.122	633.875	0.182
CAADRL	200	Greedy	60	10664.801	1124.793	0.348
Heter	100	Greedy	56	2748.612	742.014	0.290
Heter	200	Greedy	60	14509.001	973.179	0.339

depot, pickup–delivery pairs, and coordinates, but ignore time windows, capacities, vehicle-count constraints, and the original PDPTW objective.

Table 7 reports the relaxed Li&Lim evaluation results for our construction-policy baselines. The source groups 100 and 200 represent the scale of the original Li&Lim benchmark folders from which the PDP instances were converted: the 100 group yields 56 smaller instances (50–55 pickup–delivery pairs), while the 200 group yields 60 larger instances (100–107 pairs). We evaluate CAADRL and Heter on these instances.

The models are evaluated directly via greedy decoding without any training or fine-tuning on Li&Lim data. Each converted file is treated as a single-vehicle distance-only PDP instance, and Table 7 reports the average route length and inference time within each group. On the 100 source group, Heter achieves a slightly lower mean route length, while CAADRL runs faster. On the larger 200 source group, CAADRL obtains a substantially shorter mean route length than Heter (10664.801 versus 14509.001, a 26.50% reduction) with comparable execution times.

Figure 5 visualizes a representative instance from the 200 group, where CAADRL reduces the route length by 26.35% (11940.596 versus 16213.435). The comparison indicates that CAADRL tends to make fewer long detours between distant pickup and delivery regions. Together with the tabular results, this visual evidence supports the effectiveness of CAADRL in zero-shot construction-policy transfer.

5.9 Cross-Size Generalization

Beyond distribution robustness, we evaluate size generalization: whether a policy trained on one problem scale can be transferred to larger instances without retraining. Fig. 6 visualizes the greedy cross-size transfer results for models trained on PDP10/20/40/80 and tested on larger sizes.

The matched-size policy generally performs best, confirming that the learned policy adapts to scale-dependent route structure. However, cross-size degradation is moderate rather than catastrophic. On uniform instances, transferring PDP10 to PDP20, PDP20 to PDP40, and PDP40 to PDP80 increases the objective by about 4.0%, 2.2%, and 1.5%, respectively. On clustered instances, the corresponding increases are about 2.1%, 0.5%, and 1.7%. These results suggest that the learned policy captures routing principles that remain useful as the instance size grows, while still benefiting from scale-matched training.

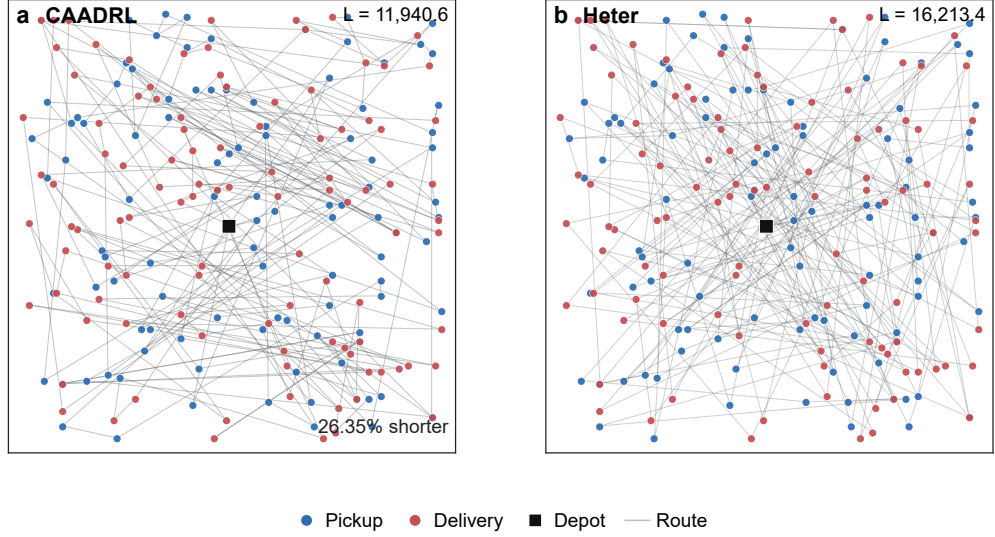


Fig. 5 Route comparison on a representative source-200 Li&Lim-PDP-relaxed instance under greedy decoding. Pickup nodes, delivery nodes, and the depot are shown in blue, red, and black, respectively. CAADRL yields a route length of 11940.596 (26.35% reduction compared with 16213.435 for Heter).

Table 8 Large-scale transfer from PDP100 to PDP200/300/500. Lower objective is better.

Distribution	Method	Decode	PDP200	PDP300	PDP500
Clustered	CAADRL	Greedy	7.785	9.912	14.118
	CAADRL	Sampling-1280	7.750	10.371	16.475
	CAADRL	Sampling-12800	7.718	10.247	16.222
	POMO	Greedy	7.798	10.116	14.484
	POMO	Sampling-1280	7.829	10.867	17.891
	POMO	Sampling-12800	7.750	10.736	17.630
Uniform	CAADRL	Greedy	16.778	21.075	28.832
	CAADRL	Sampling-1280	15.974	21.099	31.297
	CAADRL	Sampling-12800	15.765	20.798	30.917
	POMO	Greedy	16.958	21.465	28.787
	POMO	Sampling-1280	16.001	20.852	30.483
	POMO	Sampling-12800	15.816	20.568	30.058

5.10 Large-Scale Transfer from PDP100

We further evaluate a more aggressive transfer setting by training on PDP100 and directly testing on PDP200, PDP300, and PDP500. Table 8 summarizes the results for CAADRL and POMO under clustered and uniform distributions.

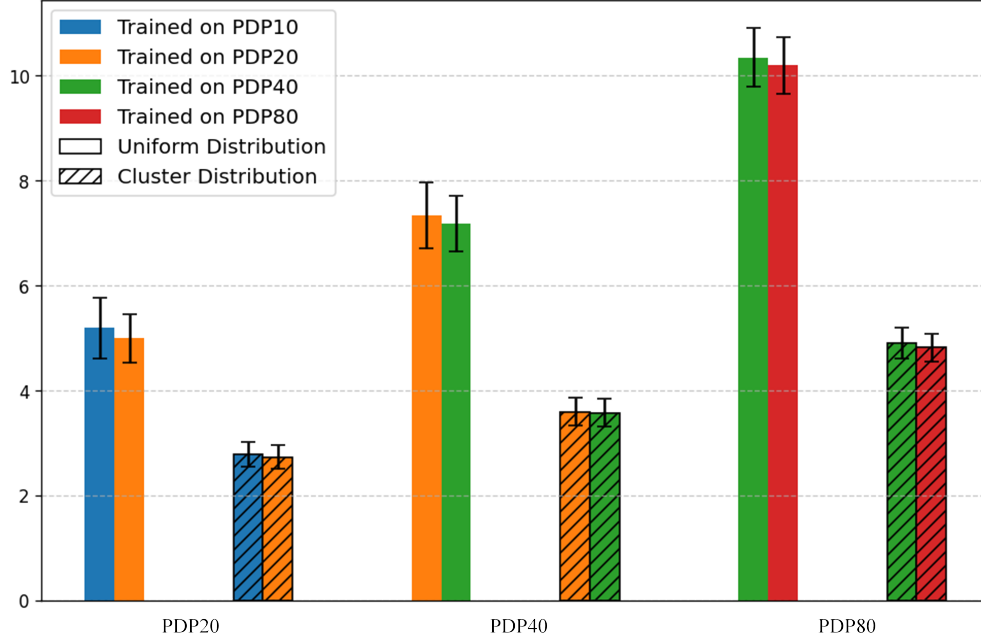


Fig. 6 Cross-size generalization of CAADRL under greedy decoding. Solid bars indicate uniform instances and hatched bars indicate clustered instances. Colors denote the training problem size. Error bars show standard deviation over test instances.

On clustered instances, CAADRL transfers effectively from PDP100 to PDP200 and remains competitive on PDP300 and PDP500. However, Table 8 shows that sampling does not always improve under extreme size extrapolation. On PDP300 and PDP500, greedy decoding is stronger than Sampling-1280 and Sampling-12800. This is consistent with the sampling-calibration diagnostics computed from the completed runs: at PDP300 and PDP500, most sampled trajectories are worse than the greedy trajectory. A policy trained on PDP100 can become less calibrated over much longer horizons; repeated sampling then draws many complete routes from a shifted distribution, whereas greedy decoding follows the highest-confidence action sequence and is more stable under the large size shift.

On uniform instances, CAADRL remains competitive with POMO. It is strongest on PDP200 under Sampling-12800, while POMO is slightly stronger on PDP300 and PDP500 under some decoding budgets. This indicates that the cluster-aware inductive bias does not prevent transfer to unstructured data, but very large uniform instances remain challenging because explicit pickup/delivery region structure becomes weak.

6 Conclusion and Future Work

This paper proposed CAADRL, a cluster-aware deep reinforcement learning framework for the Pickup and Delivery Problem. The method incorporates PDP structure

directly into a neural construction policy through a cluster-aware Transformer encoder and a hierarchical dual-branch decoder. The encoder combines global self-attention with role-based intra-cluster attention, while the decoder separately models intra-cluster continuation and inter-cluster transition preferences. The two masked probability distributions are combined by fixed probability-level average fusion, and feasibility is enforced throughout decoding by a precedence-aware dynamic mask.

Experiments show that CAADRL is effective on both clustered and uniform PDP instances. On clustered benchmarks, the model performs particularly well at medium and large scales, where local ordering within pickup/delivery regions and global transitions between regions become more important. Paired statistical tests confirm that the gains over Heter and NCS are reliable at the instance level, while route-level diagnostics show that CAADRL learns high intra-cluster continuity on clustered instances. On uniform instances, the model remains competitive and achieves the best result on the largest standard setting considered. Runtime and memory profiling further indicate that CAADRL retains low-latency construction-policy inference without iterative ruin-and-repair refinement.

The current study also has limitations. We focus on a static, single-vehicle, distance-only PDP, while many real applications involve time windows, capacities, multiple vehicles, heterogeneous fleets, stochastic travel times, and dynamic request arrivals. The main benchmarks are synthetic; the Li&Lim-PDP-relaxed experiment provides an external distribution check, but it does not constitute a full evaluation on real-world PDPTW operations. In addition, the cluster structure used in this work is role-based rather than learned from data, and large-scale transfer experiments show that extensive sampling may become less reliable when the test size is far beyond the training size.

Future work will therefore focus on three directions. First, CAADRL can be extended to richer PDP variants with time windows, capacities, multi-vehicle coordination, and dynamic requests, including applications such as synchronized microtransit transfers, drone-assisted pickup-delivery, and multi-visit drone routing [5, 22, 24]. Second, fixed role-based clusters can be replaced by learned or adaptive clusters through differentiable clustering, graph partitioning, or latent grouping modules. Third, sampling calibration and industrial deployment should be further studied, including adaptive decoding budgets, rolling-horizon control, integration with existing dispatching systems, and evaluation on large-scale operational datasets.

Overall, CAADRL demonstrates that cluster-aware representation learning and hierarchical masked decoding provide a useful modeling paradigm for neural PDP solvers. The proposed framework offers a step toward more structured, efficient, and deployable neural methods for pickup-and-delivery routing.

Declarations

Competing Interests

The authors declare that they have no conflicts of interest.

Author Contributions

Wentao Wang was responsible for drafting the manuscript and extending the original PyTorch source code to satisfy the pickup-and-delivery problem requirements. Lifeng Han contributed to the preparation of figures and was responsible for the implementation of baseline models and ablation studies. Guangyu Zou developed the original version of the reinforcement learning source code and supervised the entire study. Wentao Wang and Lifeng Han contributed equally to this work. All authors read and approved the final manuscript.

Ethical and Informed Consent for Data Used

The data used in this study were generated randomly according to predefined rules. Therefore, no ethical approval or informed consent was required.

Data Availability and Access

The data supporting the findings of this study are available from the corresponding author upon reasonable request.

References

- [1] Aerts-Veenstra M, Cherklesly M, Gschwind T (2024) A unified branch-price-and-cut algorithm for multicompartment pickup and delivery problems. *Transportation Science* 58(5):1121–1142
- [2] Arslan AM, Agatz N, Kroon L, et al (2019) Crowdsourced delivery—a dynamic pickup and delivery problem with ad hoc drivers. *Transportation Science* 53(1):222–235
- [3] Bello I, Pham H, Le QV, et al (2016) Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:161109940*
- [4] Chen X, Tian Y (2019) Learning to perform local rewriting for combinatorial optimization. *Advances in neural information processing systems* 32
- [5] Fu Z, Chow JY (2022) The pickup and delivery problem with synchronized en-route transfers for microtransit planning. *Transportation Research Part E: Logistics and Transportation Review* 157:102562
- [6] Gao C, Wei N, Walteros JL (2023) An exact approach for solving pickup-and-delivery traveling salesman problems with neighborhoods. *Transportation Science* 57(6):1560–1580
- [7] Ghilas V, Demir E, Van Woensel T (2016) An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows and scheduled lines. *Computers & Operations Research* 72:12–30

- [8] Hottung A, Tierney K (2022) Neural large neighborhood search for routing problems. *Artificial Intelligence* 313:103786
- [9] Hottung A, Bhandari B, Tierney K (2021) Learning a latent search space for routing problems using variational autoencoders. In: *International Conference on Learning Representations*
- [10] James J, Yu W, Gu J (2019) Online vehicle routing with neural combinatorial optimization and deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems* 20(10):3806–3817
- [11] Kohar A, Jakhar SK, et al (2023) Strong cutting planes for the capacitated multi-pickup and delivery problem with time windows. *Transportation Research Part B: Methodological* 176:102806
- [12] Kong D, Ma Y, Cao Z, et al (2024) Efficient neural collaborative search for pickup and delivery problems. *IEEE Transactions on Pattern Analysis and Machine Intelligence*
- [13] Kool W, Van Hoof H, Welling M (2018) Attention, learn to solve routing problems! *arXiv preprint arXiv:180308475*
- [14] Kwon YD, Choo J, et al (2020) Pomo: Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems* 33:21188–21198
- [15] Lam E, Stuckey PJ, Harabor D (2025) Optimal multi-agent pickup and delivery using branch-and-cut-and-price algorithms. *Transportation Science* 59(1):104–124
- [16] Li H, Lim A (2003) A metaheuristic for the pickup and delivery problem with time windows. *International Journal on Artificial Intelligence Tools* 12(02):173–186
- [17] Li J, Xin L, Cao Z, et al (2021) Heterogeneous attentions for solving pickup and delivery problem via deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems* 23(3):2306–2315
- [18] Li Y, Chen H, Prins C (2016) Adaptive large neighborhood search for the pickup and delivery problem with time windows, profits, and reserved requests. *European Journal of Operational Research* 252(1):27–38
- [19] Lin B, Ghaddar B, Nathwani J (2021) Deep reinforcement learning for the electric vehicle routing problem with time windows. *IEEE Transactions on Intelligent Transportation Systems* 23(8):11528–11538
- [20] Ma Y, Li J, Cao Z, et al (2022) Efficient neural neighborhood search for pickup and delivery problems. *arXiv preprint arXiv:220411399*

- [21] Ma Y, Cao Z, et al (2023) Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt. *Advances in Neural Information Processing Systems* 36:49555–49578
- [22] Meng S, Guo X, Li D, et al (2023) The multi-visit drone routing problem for pickup and delivery services. *Transportation Research Part E: Logistics and Transportation Review* 169:102990
- [23] Mourad A, Puchinger J, Chu C (2019) A survey of models and algorithms for optimizing shared mobility. *Transportation Research Part B: Methodological* 123:323–346
- [24] Mulumba T, Diabat A (2024) Optimization of the drone-assisted pickup and delivery problem. *Transportation Research Part E: Logistics and Transportation Review* 181:103377
- [25] Nazari M, Oroojlooy A, Snyder L, et al (2018) Reinforcement learning for solving the vehicle routing problem. *Advances in neural information processing systems* 31
- [26] Pacheco T, Martinelli R, Subramanian A, et al (2023) Exponential-size neighborhoods for the pickup-and-delivery traveling salesman problem. *Transportation Science* 57(2):463–481
- [27] Parragh SN, Doerner KF, Hartl RF (2008) A survey on pickup and delivery problems. *Journal für Betriebswirtschaft* 58(2):81–117
- [28] Sun Z, Yang Y (2023) Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in neural information processing systems* 36:3706–3731
- [29] Tian R, Sun Z, Chang L, et al (2025) Rapid solution for flexible pickup and delivery services problem based on improved actor-critic deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*
- [30] Vinyals O, Fortunato M, Jaitly N (2015) Pointer networks. *Advances in neural information processing systems* 28
- [31] Wang C, Cao Z, et al (2024) Deep reinforcement learning for solving vehicle routing problems with backhauls. *IEEE Transactions on Neural Networks and Learning Systems* 36(3):4779–4793
- [32] Wu Y, Song W, Cao Z, et al (2021) Learning improvement heuristics for solving routing problems. *IEEE transactions on neural networks and learning systems* 33(9):5057–5069
- [33] Wu Y, Yu Y, Wu L, et al (2026) Deep reinforcement learning approach to solving clustered vehicle routing problems. *Transportation Research Part E: Logistics*

and Transportation Review 209:104742

- [34] Xiang C, Wu Z, Tu J, et al (2024) Centralized deep reinforcement learning method for dynamic multi-vehicle pickup and delivery problem with crowdshippers. *IEEE Transactions on Intelligent Transportation Systems* 25(8):9253–9267
- [35] Zhao J, Mao M, Zhao X, et al (2020) A hybrid of deep reinforcement learning and local search for the vehicle routing problems. *IEEE Transactions on Intelligent Transportation Systems* 22(11):7208–7218